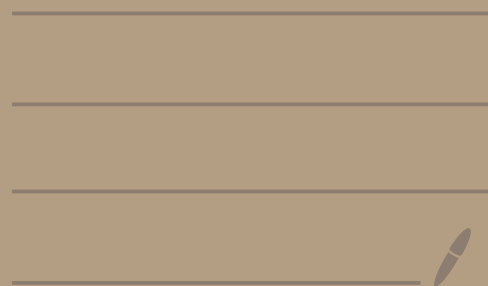


计算机系统基础 1



第一章

2. 简单回答下列问题。

- (1) 冯·诺依曼计算机由哪几部分组成？各部分的功能是什么？
- (2) 什么是“存储程序”工作方式？
- (3) 一条指令的执行过程包含哪几个阶段？
- (4) 如何划分计算机系统的层次结构？
- (5) 什么是程序的未定义行为？什么是程序的未指定行为？什么是程序的实现定义行为？
- (6) 计算机系统的用户可分为哪几类？每类用户工作在哪个层次？

(1) 冯·诺依曼计算机由哪几部分组成？各部分的功能是什么？

冯·诺依曼计算机由运算器、控制器、存储器、输入设备和输出设备5个基本部分组成。

运算器：进行算术运算、逻辑运算。

控制器：自动执行指令。控制器是计算机的指挥中心，由其“指挥”各部件自动协调地进行工作。控制器由程序计数器（PC）、指令寄存器（IR）和控制单元（CU）组成。

(PC, IR, CU)

存储器：不仅能放数据，也能存放指令，数据和指令虽然在形式上没有区别，但计算机能区分它们。存储器分为主存储器（也称内存或主存）和辅助存储器（也称外存储器或外存）。CPU能够直接访问的存储器是主存储器，辅助存储器用于帮助主存储器记忆更多的信息，辅助存储器中的信息必须调入主存储器后，才能为CPU所访问。主存储器的工作方式是按存储单元的地址进行存取，这种存取方式称为按地址存取方式。

输入设备/输出设备：操作人员通过I/O设备使用计算机。输入设备/输出设备（简称I/O设备）是计算机与外界联系的桥梁，是计算机中不可缺少的重要组成部分。

输入设备的主要功能是将程序和数以机器所能识别和接受的信息形式输入计算机。最常用也最基本的输入设备是键盘，此外还有鼠标、扫描仪、摄像机等。

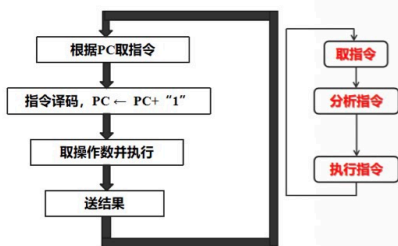
输出设备的任务是将计算机处理的结果以人们所能接受的形式或其他系统所要求的信息形式输出。最常用、最基本的输出设备是显示器、打印机。

(2) 什么是“存储程序”工作方式？

“存储程序”工作方式，规定，程序执行前，需先将程序包含的指令和数据送入主存，一旦启动程序执行，则计算机必须能够在不需操作人员干预下自动完成逐条指令的取出和执行任务。一个程序的执行就是周而复始地逐条执行指令的过程。

(3) 一条指令的执行过程包含哪几个阶段？

1. 从主存取指令、2. 对指令进行译码、3. PC增量、4. 取操作数并执行、5. 将结果送主存或寄存器保存。



(5) 什么是程序的未定义行为？什么是程序的未指定行为？什么是程序的实现定义行为？

程序的未定义行为：

指符合语言标准规范但未明确指定其结果的行为。若源程序包含未定义行为，则目标程序的每次运行结果可能不同，或者在不同平台下运行结果不同。例如：最小负整数除以-1。

程序的未指定行为：

指语言标准规范列出多种供编译器选择的行为结果，不同编译器可能选择不同的行为结果。若源程序包含未指定行为，则采用不同的编译器进行编译或采用一款编译器的不同版本进行编译，目标程序的运行结果可能不同。

例如“int i=1; i(i++);”有的编译器可能按(i++)处理，有的编译器按i(i++)处理。

程序的实现定义行为：

指语言标准规范的实现（如编译器）需要在文档中说明其选择的未指定行为。若源程序包含实现定义行为，在相同环境下运行可得到相同的结果，但将程序移植到另一个环境时，运行结果可能不同。

例如Char类型是带符号数还是无符号数整数，是实现定义行为，故程序员不应假设char是带符号数或无符号数。

(4) 如何划分计算机系统的层次结构？

传统计算机系统采用分层方式构建，即计算机系统是一个层次结构系统，通过向上层用户提供一个抽象的简洁接口而将较低层次的实现细节隐藏起来。计算机解决应用问题的过程就是不同抽象层进行转换的过程。

硬件系统和软件系统共同构成了一个完整的计算机系统。硬件是指有形的物理设备，是计算机系统中实际物理装置的总称。软件是指在硬件上运行的程序和相关的文档及数据。

把计算机系统按功能分为多级层次结构，就是有利于正确理解计算机系统的工作过程，明确软件、硬件在计算机系统中的地位和作用。其中软件：应用，算法，编程，操作系统；中间分水岭：指令集体系结构（ISA）；硬件：微体系结构，功能部件，电路，器件。

第1章 习题参考答案

• P172.

(6) 计算机系统的用户可分为哪几类？每类用户工作在哪个层次？

计算机的用户分为4类：最终用户，系统管理员，应用程序员和系统程序员。
最终用户：系统提供的简单人机交互界面和安装在计算机中的相关应用程序。
系统管理员：管理和维护计算机系统的专业人员，部分硬件层面、系统管理层面以及相关实用程序和与人交互界面。
应用程序员：实用高级程序设计语言编写程序。计算机硬件、操作系统提供的编程接口、人机交互界面和实用程序，还包括相应的程序语言处理系统。
系统程序员：开发操作系统、编译器和实用程序等系统软件。需要熟悉计算机底层的相关硬件。



• P172.

(7) 应用程序二进制接口 (ABI) 和应用程序接口 (API) 各自与哪类计算机系统用户关系最密切 (即哪类用户直接使用ABI和API标准) ?

应用程序二进制接口 (ABI) 与应用程序接口 (API) 分别与不同类型的计算机系统用户关系最密切:

ABI是为运行在特定ISA及特定操作系统上的应用程序规定的一种机器级目标代码层接口, 包含了运行在特定ISA及特定操作系统上的应用程序所对应的目标代码生成时必须遵循的约定。

ABI与用户关系: ABI主要面向 **底层开发人员和系统集成者**, 涉及硬件和操作系统层面的二进制兼容性。

API定义了较高层次的源程序代码和库之间的接口, 通常是与硬件无关的接口。

API与用户关系: API主要面向 **应用开发人员和软件工程师**, 提供高级编程接口。

两者的核心区别在于: ABI关注底层二进制兼容性, API关注高层功能封装。

5. 图 1.1 所示的模型机 (采用图 1.2 所示的指令格式) 的指令系统中, 除了有 `mov (op=0000)`、`add (op=0001)`、`load (op=1110)` 和 `store (op=1111)` 指令外, R 型指令还有 `sub (op=0010)` 和 `mul (op=0011)` 等指令, 请仿照图 1.3 给出求解表达式 “ $z=(x-y)*y;$ ” 所对应的指令序列 (包括机器代码和对应的汇编指令) 以及在主存中存放的内容, 并仿照图 1.5 给出每条指令的执行过程以及所包含的微操作。

P18 第5题 参考答案: 看清每条指令功能 “ $z=(x-y)*y$ ” 所对应的指令队列

存储单元 (根据下面汇编指令条数): $x:6\#$ $y:7\#$ $z:8\#$

汇编指令	机器指令
<code>load r0,6#</code>	<code>1110 0110</code>
<code>mov r1,r0</code>	<code>0000 0100</code>
<code>load r0,7#</code>	<code>1110 0111</code>
<code>sub r1,r0</code>	<code>0010 0100</code>
<code>mul r0,r1</code>	<code>0011 0001</code>
<code>store 8#,r0</code>	<code>1111 1000</code>

<code>load</code>	<code>1110</code>	取数
<code>mov</code>	<code>0000</code>	移动
<code>sub</code>	<code>0010</code>	减
<code>mul</code>	<code>0011</code>	乘
<code>add</code>	<code>0001</code>	加
<code>store</code>	<code>1111</code>	存储

图1.5每条指令的执行过程及对应的微操作可以仿写出来 (不要求)。

格式	4 位	2 位	2 位	功能说明	汇编指令
R 型	0000	rt	rs	$R[rt] \leftarrow R[rs]$	<code>mov rt, rs</code>
R 型	0001	rt	rs	$R[rt] \leftarrow R[rt] + R[rs]$	<code>add rt, rs</code>
R 型	0010	rt	rs	$R[rt] \leftarrow R[rt] - R[rs]$	<code>sub rt, rs</code>
R 型	0011	rt	rs	$R[rt] \leftarrow R[rt] * R[rs]$	<code>mul rt, rs</code>
M 型	1110	addr		$R[0] \leftarrow M[addr]$	<code>load r0, addr#</code>
M 型	1111	addr		$M[addr] \leftarrow R[0]$	<code>store addr#, r0</code>

第二章

(3) 单精度浮点数减法指令

- ⑧ 假定机器 M 的字长为 32 位，用补码表示带符号整数。表 2.12 中第一列给出了在机器 M 上执行的 C 程序中的关系表达式，请参照已有的表栏内容完成表中后三栏内容的填写。

表 2.12 题 8 表

关系表达式	运算类型	结果	说明
0 == 0U			
-1 < 0			
-1 < 0U			
2147483647 > -2147483647 - 1	无符号整数	0	$11 \dots 1B (2^{32}-1) > 00 \dots 0B (0)$
2147483647U > -2147483647 - 1	带符号整数	1	$011 \dots 1B (2^{31}-1) > 100 \dots 0B (-2^{31})$
2147483647 > (int) 2147483648U			
-1 > -2			
(unsigned) -1 > -2			

P64 第8题 参考答案:

若同时有无符号和带符号整数，则 C 编译器将带符号整数强制转换为无符号数

关系表达式	类型	结果	说明
0 == 0U	无	1	$00 \dots 0B = 00 \dots 0B$
-1 < 0	带	1	$11 \dots 1B (-1) < 00 \dots 0B (0)$
-1 < 0U	无	0*	$11 \dots 1B (2^{32}-1) > 00 \dots 0B (0)$
2147483647 > -2147483647 - 1	带	1	$011 \dots 1B (2^{31}-1) > 100 \dots 0B (-2^{31})$
2147483647U > -2147483647 - 1	无	0*	$011 \dots 1B (2^{31}-1) < 100 \dots 0B (2^{31})$
2147483647 > (int) 2147483648U	带	1*	$011 \dots 1B (2^{31}-1) > 100 \dots 0B (-2^{31})$
-1 > -2	带	1	$11 \dots 1B (-1) > 11 \dots 10B (-2)$
(unsigned) -1 > -2	无	1	$11 \dots 1B (2^{32}-1) > 11 \dots 10B (2^{32}-2)$

有符号 → 无符号

当你把一个 int 强制转换为 unsigned int 时，公式如下：

$$T2U_w(x) = \begin{cases} x + 2^w, & x < 0 \\ x, & x \geq 0 \end{cases}$$

eg: $1 \rightarrow 1$
 $-1 \rightarrow 2^{32}-1$

无符号 → 有符号

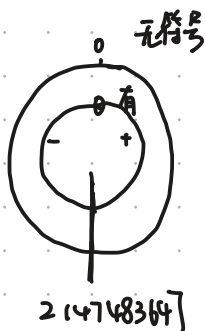
3. 无符号数 → 有符号数 (U2T)

当你把一个 unsigned int 强制转换为 int 时，逻辑正好反过来：

$$U2T_w(u) = \begin{cases} u, & u \leq TMax_w \\ u - 2^w, & u > TMax_w \end{cases}$$

eg: $1 \rightarrow 1$

$T_{max} = 2^{w-1} - 1$ $2^{31}-1 = 2147483647$
 $T_{min} = -2^{w-1}$ $-2^{31} = -2147483648$



- ⑨ 在 32 位计算机中运行一个 C 程序，该程序中出现了以下变量的初值，请写出它们对应的机器数 (用十六进制表示)。
- (1) int x = -32768 (2) short y = 522 (3) unsigned z = 65530
 (4) char c = '@' (5) float a = -1.1 (6) double b = 10.5

P65 第9题 参考答案: 注: 32位计算机

转为2进制

- (1) -32768 = $-2^{15} = -1000\ 0000\ 0000\ 0000$ 符号扩展为 FFFF8000H. int 4个字节 → 8位

- (2) 522 = 10 0000 1010 所以 y = 0000 0010 0000 1010 = 020AH. 0000 020AH

- (3) 65530 = $2^{16} - 1 - 5 = 1111\ 1111\ 1111\ 1010$ 所以 = 0000FFFAH.

- (4) @ 的 ASCII 码为 40H.

- (5) 尾数为 -1.1 = -1.000 11 0011 0011 0011..... [0011] 循环 = -1.000 1100 1100 1101
 = -1.000 1100 1100 1100 1100 1100 1100 1100 舍入处理后最低位加1
 = -1.000 1100 1100 1100 1101

阶码为 0+127=0111 1111 127 C C C
 所以机器数为: 1 0111 1111 000 1100 1100 1100 1101 = BF8CCCCDH

- (6) 10.5 = 1010.1B = 1.0101 * 2³
 所以 阶码为 3+1023=100 0000 0010
 故: 机器数为 0 100 0000 0010 0101 0000 0000.....
 = 4025 0000 0000 0000H

2 | 1026
 2 | 513 0
 2 | 256 1
 2 | 128 0
 2 | 64 0
 2 | 32 0
 2 | 16 0
 2 | 8 0
 2 | 4 0
 2 | 2 0
 2 | 1 0
 0 1

总结 求一个数的机器码

1. 整数:

① 正数 10进制转2进制, 高位添0

② 负数 10进制转2进制, 取反加1, 添符号

2. 浮点数:

① 单精度 (32位) $1 + 8 + 23$ 偏置常数 127

② 双精度 (64位) $1 + 11 + 52$ 偏置常数 1023

P65 第10题 参考答案: 注: 32位计算机

(1) $x = -1111\ 1111\ 1111\ 1010 = -(65535-5) = -65530$

(2) $DFCH = -010\ 0000\ 0000\ 0100 = -(8192+4) = -8196$

(3) $FFFF\ FFAH = 1\dots1\ 1010 = (2^{32}-1) - 5 = 2^{32}-6$

(4) $2AH = 0010\ 1010 = 42$ 若为ASCII码则表示字符**

(5) $C4480000H = 1100\ 0100\ 0100\ 1000\ 0\dots0$
阶码为 1000 1000, 所以阶码为 $136-127=9$
尾数为 -1.1001 所以 $a = -1.1001 * 2^9 = -800$

(6) $C024800000000000H = 1100\ 0000\ 0010\ 0100\ 1000\ 0\dots0$
阶码 100 0000 0010, 所以阶码为 $1026-1023=3$
尾数为 -1.01001 所以 $a = -1.01001 * 2^3 = -10.25$

机器码求真值

18. 假设以下 C 语言函数 `compare_str_len()` 用来判断两个字符串的长度, 当字符串 `str1` 的长度大于 `str2` 的长度时函数返回值为 1, 否则为 0。

```
1 int compare_str_len(char *str1, char *str2)
2 {
3     return strlen(str1) - strlen(str2) > 0; X return strlen(str1) > strlen(str2)
4 }
```

已知 C 标准库函数 `strlen()` 原型声明为 “`size_t strlen(const char *s);`”, 其中, `size_t` 定义为 `unsigned int` 类型。请问: 函数 `compare_str_len()` 在什么情况下返回结果不正确? 为什么? 为使函数正确返回结果, 应如何修改代码?

因为 `size_t` 被定义为 `unsigned int`, 所以库函数 `strlen` 返回值为无符号整数。

而函数 `compare_str_len` 中的返回值为 `strlen(str1) - strlen(str2) > 0`, 这个表达式左侧是两个无符号数相减, 差还是无符号数, 因此总是大于等于 0, 所以在即使 `str2` 大于 `str1` 长度时, 也返回 1, 因此是错误的。

因此第 3 行改为:

```
return strlen(str1) > strlen(str2)
```

21) 以下 C 函数 arith() 是直接写用 C 语言写的, 而 optarith() 是对 arith() 函数以某个确定的 M 和 N 编译生成的机器代码反编译生成的。根据 optarith() 推断函数 arith() 中 M 和 N 的值各是多少, 并推断编译器对 arith() 函数做了哪些优化。

```

1 #define M
2 #define N
3 int arith(int x, int y)
4 {
5     int result = 0;
6     result = x*M + y/N;
7     return result;
8 }

```

```

10 int optarith ( int x, int y)
11 {
12     int t = x;
13     x <<= 4;
14     x = t;
15     if ( y < 0 ) y += 3;
16     y >>= 2;
17     return x+y;
18 }

```

等价

M=15

如果 y<0, 加上偏置量 2^k-1=3, 所以 N=4

P67 第21题 参考答案

对于反编译结果进行分析: 对于 x, 有一条“x左移4位”, 即: $x=16x$
 还有一条“减法”, 即: $x=16x-x=15x$
 再根据源程序可知 M=15.

对于 y, 有一条“y右移2位”, 即: $y=y/4$ 根据源程序可知 N=4, 但是当 y<0, 执行 $y>>2$ 后的值不等于 $y/4$, 例如 $y=-1$ 执行 $y>>2$, 因为 y 全为 1, 右移2位后还是全 1, 即: $-1>>2=-1$; 而原函数 arith 中 $y/4$ 则为 0 ($-1/4$)

对于带符号整数, 采用算术右移, 高位补符号, 低位移出。
 对于符号位为 0, 与无符号数相同, 采用移位与直接相除得到的商完全一样。
 对于符号位为 1, 若低位移出的是非全 0, 则说明不能整除, 例如 $-3/2$, (假定采用 4 位补码), $1101>>1=1110.1$ (小数点后面部分移出) 得到了商 1110=-2, 而精确商是 -1.5, 即整数商应该为 -1。这种情况下, 移位得到的商与直接相除得到的商不一样, 需要进行校正。
 校正方法: 对于带符号数, 若 $x<0$, 则在右移前, 先将 x 加上偏置量 2^{k-1} , 然后再右移 k 位。例如: optarith 中, if (y<0) y+=3, 以对 y 进行纠正。

22) 下列几种情况所能表示的数的范围是什么?

- (1) 16 位无符号整数
- (2) 16 位原码定点小数

P67 第22题 参考答案

- (1) 16 位无符号整数范围: $0 \sim 2^{16}-1$, 即 0~65535
- (2) 16 位原码定点小数范围: $-(1-2^{-15}) \sim +(1-2^{-15})$
- (3) 16 位移码定点整数范围: (偏置常数为 32768, 2^{15})
 因为 16 位移码的表示范围是: $0 \sim 2^{16}-1$ (即 0 到 65535)
 根据移码值=真值+偏置常数, 所以把移码值转换为真值:
 真值=移码值-32768
 因此最小值: $0-32768=-32768$
 最大值: $65535-32768=32767$
 所以 16 位移码定点整数范围 (偏置常数为 32768): $-32768 \sim +32767$ (符号位与补码相反, 无+32768 编码)
- (4) 16 位移码定点整数范围 (偏置常数为 32767, $2^{15}-1$)
 即 $32767 \sim +32768$ (编码对称分布, 支持+32768, 但无-32768)
- (5) 16 位补码定点整数范围: $-32768 \sim +32767$
- (6) 下述格式浮点数范围: 关于原点对称的, 移码的偏置常数为 128
 最大正数: $+0.111\ 1111 \times 2^{1111\ 1111} = +(1-2^{-7}) \times 2^{127}$
 最小正数: $+0.100\ 0000 \times 2^{0000\ 0000} = +2^{-1} \times 2^{-128} = +2^{-129}$
 最大负数: $-0.100\ 0000 \times 2^{0000\ 0000} = -2^{-1} \times 2^{-128} = -2^{-129}$
 最小负数: $-0.111\ 1111 \times 2^{1111\ 1111} = -(1-2^{-7}) \times 2^{127}$

- (3) 16 位移码定点整数 (偏置常数为 32768)
- (4) 16 位移码定点整数 (偏置常数为 32767)
- (5) 16 位补码定点整数
- (6) 下述格式的浮点数 (基为 2, 移码的偏置常数为 128)

数符	阶码	尾数
1 位	8 位移码	7 位原码小数数值部分

23) 以 IEEE 754 单精度浮点数格式表示下列十进制数。

+1.75, +19, -1/8, 258

- (1) $+1.75 = +1.11 \times 2^0$, 所以, 符号 s=0, 阶码 e=0+127=127=0111 1111
 尾数 f=1.110...0 所以 IEEE 754 单精度表示为: 0 0111 1111 110 0000 0000
 0000 0000 0000 = 3FE0 0000H
- (2) $+19 = +10011 = +1.0011 \times 2^4$, 所以, 符号 s=0, 阶码 e=4+127=127=1000 0011
 尾数 f=1.00110...0 所以 IEEE 754 单精度表示为: 0 1000 0011 001 1 000
 0000 0000 0000 0000 = 4198 0000H
- (3) $-1/8 = -0.125 = -0.001 = -1.0 \times 2^{-3}$, 符号 s=1, 阶码 e=-3+127=124=0111 1100
 尾数 f=1.0...0 所以 IEEE 754 单精度表示为: 1 0111 1100 000 0000 0000
 0000 0000 0000 = BE00 0000H
- (4) $258 = 100000010 = 1.0000001 \times 2^8$, 符号 s=0, 阶码 e=8+127=135=1000 0111
 尾数 f=1.0000001 所以 IEEE 754 单精度表示为: 0 1000 0111 0000001 0000
 0000 0000 0000 = 4381 0000H

[illegible]

31 对于 2.6.5 节中例 2.31 存在的整数溢出漏洞，如果将其中的第 5 行改为以下两个语句：

```
unsigned long long arraysize=count*(unsigned long long)sizeof(int);
int *myarray = (int *) malloc(arraysize);
```

已知 C 标准库函数 malloc() 的原型声明为 “void *malloc(size_t size);”，其中，size_t 定义为 unsigned int 类型，则上述改动能否消除整数溢出漏洞？若能则说明理由；若不能则给出修改方案。

P69 第31题 参考答案

仍无法消除整数溢出漏洞，这种改动方式虽然使得 arraysize 的表示范围扩大了，但是调用 malloc 函数时，若 arraysize 的值大于 32 位的 unsigned int 最大可表示值，则 malloc 函数只能按照 32 位数给出的值去申请空间，同样会发生整数溢出漏洞。

消除方法：在调用 malloc 之前检测所申请的空间是否大于 32 位无符号整数的表示范围，若是，则返回 -1，否则，再申请空间并继续进行数组复制。具体程序改为如下：

```
int copy_array(int *array, int count){
    int i;
    /*在堆区申请一块内存*/
    unsigned long long arraysize=count*(unsigned long long) sizeof(int);
    size_t myarraysize=(size_t) arraysize;
    if (myarraysize!=arraysize)
        return -1;
    int *myarray=(int *) malloc(myarraysize);
    if (myarray=NULL)
        return -1
    for (i=0;i<count; i++)
        myarray[i]=array[i];
    return count;
}
```

4. 以下是一组关于浮点数按位级进行运算的编程题目，其中用到一个数据类型 float_bits，它被定义为 unsigned int 类型。以下程序代码必须采用 IEEE 754 标准规定的运算规则，例如，舍入应采用就近舍入到偶数的方式。此外，代码中不能使用任何浮点数类型、浮点数运算和浮点常数，只能使用 float_bits 类型；不能使用任何复合数据类型，如数组、结构体和联合等；可以使用无符号整数或带符号整数的数据类型、常数和运算。要求编程实现以下功能并进行正确性测试。

(1) 计算变量 f 的绝对值 |f|。若 f 为 NaN，则返回 f，否则返回 |f|。函数原型如下：

```
float_bits float_abs(float_bits f);
```

(2) 计算变量 f 的负数 -f。若 f 为 NaN，则返回 f，否则返回 -f。函数原型如下：

```
float_bits float_neg(float_bits f);
```

(3) 计算 0.5*f。若 f 为 NaN，则返回 f，否则返回 0.5*f。函数原型如下：

```
float_bits float_half(float_bits f);
```

(4) 计算 2.0*f。若 f 为 NaN，则返回 f，否则返回 2.0*f。函数原型如下：

```
float_bits float_twice(float_bits f);
```

(5) 将 int 型变量 i 的位序列转换为 float 型位序列。函数原型如下：

```
float_bits float_i2f(int i);
```

(6) 将变量 f 的位序列转换为 int 型位序列。若 f 为非规格化数，则返回值为 0；若 f 是 NaN 或 $\pm \infty$ 或超出 int 型数可表示范围，则返回值为 0x8000 0000；若 f 带小数部分，则考虑舍入。函数原型如下：

```
int float_f2i(float_bits f);
```

P70 第41题 参考答案

```
(1) 计算浮点数f的绝对值|f|,若f为NaN, 则返回f, 否则返回|f|。
float_bits float_abs(float_bits f) {
    unsigned sign=f>>31;
    unsigned exp=f>>23&0xFF;
    unsigned frac=f&0x7FFFFFFF;
    if (exp==0xFF) && (frac!=0) || (sign==0) /*f为NaN或正数*/
        return f;
    else
        return f&0x7FFFFFFF; /*f为负数*/
}
```

P70 第41题 参考答案

```
(3) 计算0.5f, 若f为NaN, 则返回f, 否则返回0.5f。
float_bits float_half(float_bits f) {
    unsigned sign=f>>31;
    unsigned exp=f>>23&0xFF;
    unsigned frac=f&0x7FFFFFFF;
    if (exp==0xFF) && (frac!=0) /*f为NaN*/
        return f;
    else if ((exp==0)||((exp==0xFF) && (frac==0)) /*f为0或∞*/
        return f;
    else if (exp==0) && (frac!=0) /*f为非规格化数*/
        return sign<<31| frac>>1;
    else /*f为规格化数*/
        exp=exp+0xFF;
        if (exp!=0) /*0.5f为规格化数*/
            return sign<<31| exp<<23| frac;
        else /*0.5f为非规格化数*/
            return sign<<31| (frac| 0x800000)>>1;
    }
}
```

```
(5) 将int型整数i的位序列转换为float型位序列。
float_bits float_i2f(int i) {
    unsigned pre_count=30;
    unsigned sign=(unsigned) i>>31;
    unsigned neg_i;
    if (i==0) /*i为0*/
        return i;
    if (sign==0) /*i为正数*/
        while (i>>pre_count==0) pre_count--;
    return sign<<31| (127+pre_count)<<23| (unsigned)(i<<(32-pre_count))>>23;
    }
    else /*i为负数*/
        while (i<<pos_count==0) pos_count--;
        neg_i=~(i>>(32-pos_count))<<(32-pos_count)|(1<<(31-pos_count));
        while (neg_i>>pre_count==0) pre_count--;
        return sign<<31| (127+pre_count)<<23| neg_i<<(32-pre_count)>>23;
    }
}
```

P70 第41题 参考答案

```
(2) 计算浮点数f的负数-f, 若f为NaN, 则返回f, 否则返回-f。
float_bits float_neg(float_bits f) {
    unsigned exp=f>>23&0xFF;
    unsigned frac=f&0x7FFFFFFF;
    if (exp==0xFF) && (frac!=0) /*f为NaN*/
        return f;
    else
        return f^0x80000000; /*^是按位异或*/
}
```

P70 第41题 参考答案

```
(4) 计算2.0*f, 若f为NaN, 则返回f, 否则返回2.0*f。
float_bits float_twice(float_bits f) {
    unsigned sign=f>>31;
    unsigned exp=f>>23&0xFF;
    unsigned frac=f&0x7FFFFFFF;
    if (exp==0xFF) && (frac!=0) /*f为NaN*/
        return f;
    else if ((exp==0)||((exp==0xFF) && (frac==0)) /*f为0或∞*/
        return f;
    else if (exp==0) && (frac!=0) /*f为非规格化数*/
        if (frac&0x400000) /*f的尾数第一位为1*/
            return sign<<31| 1<<23| (frac&0x3FFFFFF)<<1;
        else /*f的尾数第一位为0*/
            return sign<<31| frac<<1;
        return sign<<31| frac>>1;
    }
    else /*f为规格化数*/
        exp=exp+0x01;
        if (exp!=0xFF) /*2.0f为规格化数*/
            return sign<<31| exp<<23| frac;
        else /*2.0f为非规格化数*/
            return sign<<31| exp<<23;
    }
}
```

P85 第41题 参考答案

(6) 将float型位序列转换为int型整数i的位序列。若f为非规格化数, 则返回值为0; 若f是NaN或±∞或超出int范围, 则返回值为0x80000000;若f带小数部分, 则考虑舍入。

```
int float_f2i(float_bits f) {
    unsigned sign=f>>31;
    unsigned exp=f>>23 & 0xFF;
    unsigned frac=f&0x7FFFFFFF;
    unsigned exp_value=exp-127;
    unsigned neg_i;
    unsigned pos_count=31;
    if ((exp==0xFF) || (exp_value>30)) /*f为NaN或∞或值太大*/
        return 0x80000000;
    else if ((exp==0) || (exp_value<0)) /*f为非规格化数或0或值太小*/
        return 0;
    else if (sign==0) /*f为正的规格化数*/
        return (1<<30| (frac<<7)>>(30-exp_value));
    else /*f为负的规格化数*/
        neg_i=(1<<30| (frac<<7)>>(30-exp_value));
        while (neg_i<<pos_count==0) pos_count--;
        return ~(neg_i>>(32-pos_count))<<(32-pos_count)|(1<<(31-pos_count));
    }
}
```

第三章

③对于以下 AT&T 格式汇编指令，根据操作数的长度确定对应指令助记符中的长度后缀，并说明每个操作数的寻址方式。

(1) mov 8(%ebp, %ebx, 4), %eax
(2) mov %al, 12(%ebp)
(3) add (, %ebx, 4), %eax
(4) or (%ebx), %dh
(5) push %rcx

P118 第3题 参考答案:

- ((1))后缀:w, 源: 基址+比例变址+偏移 目的: 寄存器
((2))后缀:b, 源: 寄存器 目的: 基址+偏移
((3))后缀:l, 源: 比例变址 目的: 寄存器
((4))后缀:b, 源: 基址 目的: 寄存器
((5))后缀:l, 源: 立即数 目的: 栈 32位IA-32
((6))后缀:l, 源: 立即数 目的: 寄存器
((7))后缀:q, 源: 寄存器 目的: 寄存器
((8))后缀:l, 源: 基址+变址+偏移 目的: 寄存器

第3章 程序转换与指令系统

(6) mov \$0xFFF0, %eax
(7) test %rax, %rax
(8) lea 8(%ebx, %esi), %eax

④使用汇编器处理以下 IA-32/x86-64 中 AT&T 格式代码时都会产生错误，请说明每一行代码存在什么错误。

(1) movl \$0xFF, (%eax)
(2) movb %ax, 12(%ebp)
(3) addl %ecx, \$0xF0
(4) orw \$0xFFFF, (%ebx)
(5) addb \$0xF8, (%al)
(6) movl %bx, %eax
(7) andl %esi, %eax
(8) movq 8(%ebp, , 4), %rax
(9) leaq 20(%rdi, %rsi), %eax

P119 第4题 参考答案:

- ((1))源操作数是立即数0xFF, 需要在前面加"\$".
((2))源操作数是16位, 而长度后缀是字节"b", 不一致.
((3))目的操作数不能是立即数.
((4))操作数位数超过16位, 而长度后缀为16位的"w".
((5))不能用8位寄存器作为目的操作数地址所在寄存器.
((6))源操作数寄存器与目的操作数寄存器长度不一致.
((7))不存在ESX寄存器.
((8))源操作数地址中缺少变址寄存器.
((9))两个变址寄存器同时存在, 目的操作数寄存器不是64位

⑤假设变量 x 和 ptr 的类型声明如下:

```
src_type x;  
dst_type *ptr;
```

这里, src_type 和 dst_type 是用 typedef 声明的数据类型。有以下 C 语言赋值语句:

```
*ptr=(dst_type) x;
```

若 x 在寄存器 EAX 或 AX 或 AL 中, ptr 在寄存器 EDX 中, 则对于表 3.11 中给出的 src_type 和 dst_type 类型组合, 写出实现上述赋值语句的 IA-32 机器级代码, 要求用 AT&T 格式汇编指令表示。

表 3.11 题 5 表

src_type	dst_type	机器级表示
char	int	
int	char	
int	unsigned	
short	int	
unsigned char	unsigned	
char	unsigned	
int	int	

P119 第5题 参考答案:

注: MOVS, MOVZ 目的操作数只能是寄存器。

- ((1)) movsbl %al, %ebx
movl %ebx, (%edx)
((2)) movb %al, (%edx)
((3)) movl %eax, (%edx)
((4)) movswl %ax, %ebx
movl %ebx, (%edx)
((5)) movzbl %al, %ebx
movl %ebx, (%edx)
((6)) movzbl %al, %ebx
movl %ebx, (%edx)
((7)) movl %eax, (%edx)

⑥假设变量 x 和 y 分别存放在寄存器 RAX 和 RCX 中, 给出以下各指令执行后寄存器 RDX 中的结果。

(1) leaq (%rax), %rdx
(2) leaq 4(%rax, %rcx), %rdx
(3) leaq (%rax, %rcx, 8), %rdx
(4) leaq 16(%rcx, %rax, 2), %rdx
(5) leaq (, %rax, 4), %rdx
(6) leaq (%rax, %rcx), %rdx

P119 第6题 参考答案:

- ((1)) R[edx]=x
((2)) R[edx]=x+y+4
((3)) R[edx]=x+8*y
((4)) R[edx]=y+2*x+16
((5)) R[edx]=4*x
((6)) R[edx]=x+y

假设在 IA-32 系统中以下地址以及寄存器中存放的机器数如表 3.12 所示。

表 3.12 题 7 表

地址	机器数	寄存器	机器数
0x0804 9300	0xffff ff0	EAX	0x0804 9300
0x0804 9400	0x8000 0008	EBX	0x0000 0100
0x0804 9384	0x80f7 ff00	ECX	0x0000 0010
0x0804 9380	0x908f 12a8	EDX	0x0000 0080

分别说明执行以下指令后，哪些存储单元地址或寄存器中的内容会发生改变？改变后的内容是什么？

计算机系统：基于 x86+Linux 平台

120

么？条件标志 OF、SF、ZF 和 CF 会发生什么改变？

- (1) `addl (%eax), %edx`
- (2) `subl (%eax, %ebx), %ecx`
- (3) `orw 4(%eax, %ecx, 8), %bx`
- (4) `testb $0x80, %dl`
- (5) `imull $32, (%eax, %edx), %ecx`
- (6) `mulw %bx`
- (7) `decw %cx`

• (1) `addl` & (2) `subl`：典型的二元算术运算。需要将内存中的值与寄存器值相加/减。这里考察的是补码运算，以及通过补码结果来倒推四个标志位。

• (7) `decw`：16 位自减。记住它不影响 CF，这在循环计数中非常重要。

逻辑与测试指令

• (3) `orw`：字长 (16 位) 逻辑或。注意它对标志位的特殊处理 (CF/OF 清零)。

• (4) `testb`：字节长 (8 位) 逻辑与。test 与 and 的区别在于 test 不改变目的寄存器的值，只改标志位。常用于检查某一位是否为 1。

乘法指令 (最复杂的计算)

• (5) `imull` (三操作数)： `imull $32, (%eax, %edx), %ecx`。

• 计算：存储值 * 32。

• 结果存入 %ecx。

• 标志位：如果结果不能被 32 位容纳 (即符号扩展后不等于原值)，则 $CF = OF = 1$ 。

• (6) `mulw %bx` (隐式操作数)：无符号乘法。

• 计算： $AX * BX$ 。

• 结果：高 16 位存入 DX，低 16 位存入 AX。

• 标志位：如果结果的高 16 位 (DX) 不为 0，则 $CF = OF = 1$ 。

P119 第7题 参考答案：

(1) 指令功能是 $R[edx] \leftarrow R[edx] + M[R[ecx]] = 0x00000080 + M[0x8049300]$
寄存器 EDX 会改变内容，改变后为： $0x00000080 + FFFFFFFF0H = (1)00000070H$ 。
因此 EDX 的内容改变为 $0x00000070H$ 。加法指令影响这几个标志，各标志位分别为：OF=0 ZF=0 SF=0 CF=1

(2) 指令功能是 $R[ecx] \leftarrow R[ecx] - M[R[ecx] + R[ebx]] = 0x00000010 + M[0x8049400]$
寄存器 ECX 会改变内容，改变后为： $0x00000010 - 80000008H = 0x00000010 + 7FFFFFFFH = (0)80000008H$
因此 ECX 的内容改变为 $0x80000008H$ 。减法指令影响这几个标志，各标志位分别为：OF=1 ZF=0 SF=1 CF=1

(3) 指令功能是 $R[bx] \leftarrow R[bx] \text{ or } M[R[ecx] + R[ecx] * 8 + 4] = 0x0100 \text{ or } M[0x8049384] = 0x0100 \text{ or } FF00H = FF00H$

寄存器 BX 会改变内容，改变后为： $0xFF00$ 。Or 指令执行后 OF=CF=0, ZF 和 SF 根据结果设置，因为结果不为 0，所以 ZF=0；最高位为 1，所以 SF=1。

(4) Test 指令不改变任何寄存器内容，但会根据“与”操作改变标志。因为 $R[dl]$ and $0x80 = 80H$ and $80H = 80H$ 。Test 执行后 OF=CF=0, ZF 和 SF 根据结果设置，因为结果不为 0，所以 ZF=0；最高位为 1，所以 SF=1。

P119 第7题 参考答案： (续)

(5) 指令功能是 $R[cx] \leftarrow M[R[ecx] + R[edx]] * 32$ ，即存储单元 $0x8049380$ 中的内容 $0x908F12A8$ 与立即数 32 相乘，最后将乘积的低 32 位送存储单元 $0x8049380$ 中。(可以将 $0x908F12A8$ 符号扩展成 64 位，然后左移 5 位) 得到低 32 位 $= 0x11E25500$ 。因为高 32 位不是全 0 ($0x12$)，也不全是 1，所以 $OF=CF=1$ 。(imul 3 个显式操作数 参见 P100 解释)

(6) 指令功能是 $R[dx-ax] \leftarrow R[ax] * R[bx]$ 即将寄存器 AX 和 BX 中的内容相乘，其乘积的高 16 位存入 DX 中，低 16 位存入 AX 中。

因为 $R[ax] = 0x9300$ $R[bx] = 0x0100 = 2^8$ ，相当于把 $9300H$ 左移 8 位。因此 $dx = 0x0093$ ax 全部为 0。因为高 16 位不是全 0，因此 $OF=CF=1$ 。

(7) 指令功能是 $R[cx] \leftarrow R[cx] - 1$ ，因为 $R[cx] = 0x0010$ ， $0010 - 0001 = 0010 + FFFFH = (1)000FH$ ，DEC 指令会影响标志 OF、ZF 和 SF，所以 $OF=0, ZF=0, SF=0$ 。(参见 P101 的表格 3.8)

8. 已知 IA-32 采用小端方式，根据给出的 IA-32 代码反汇编结果 (部分信息用 x...x 表示) 回答问题。

(1) 已知 je 指令的操作码为 0111 0100，je 指令的跳转目标地址是什么？call 指令中的跳转目标地址 $0x80483b1$ 是如何反汇编出来的？

je 指令占 5 个字节
call 指令占 5 个字节
je 0xxxxxx
call 0x80483b1<test>

(2) 已知 jb 指令的操作码为 0111 0010，jb 指令的跳转目标地址是什么？movl 指令中的目的地址是如何反汇编出来的？

jb 0xxxxxx
movl \$0x1, 0x804a800

(3) 已知 jle 指令的操作码为 0111 1110，jle 和 mov 指令的地址各是什么？

jle 0x80492e0
mov %edx, %eax

(4) 已知 jmp 指令的跳转目标地址采用相对寻址方式，jmp 指令操作码为 1110 1001，其跳转目标地址是什么？

jmp
sub %eax, %edx

P120 第8题 参考答案：

(1) 条件转移都是 目标地址=PC+偏移量

因为 je 的操作码是 01110100，所以机器代码中 7408H 中的 08H 为偏移量，所以转移目标地址为： $0x804838c + 2 + 0x8 = 0x8048396$
call 中的目标地址 $0x80483b1 = 0x804838c + 5 + 0x1e$ ，由此看出，call 指令机器代码中后面的 4 个字节为偏移量，因为 IA-32 采用小端方式，故偏移量为 $0000001EH$ 。Call 指令本身占 5 个字节，所以 $PC = 0x804838c + 5$

(2) jb 指令中 F6H 为偏移量，故其转移地址为： $0x8048390 + 2 + 0xF6 = 0x8048388$
计算过程： $F6H$ ，真值是 -10， $0x8048392 - 10 = 0x8048388$

或者符号扩展后相加： $0x8048390 + 2 + 0xFFFFF6 = 0x8048388$
movl 指令中的机器代码有 10 个字节，前 2 个字节是操作码，后面 8 个字节为两个立即数，因为是小端方式，所以第一个立即数为 $0804A800H$ ，即汇编指令中的目的地址 $0x804a800$ ，最后 4 个字节为立即数 $00000001H$ ，即汇编指令中的常数 $0x1$ 。

(3) jle 指令中的 7EH 为操作码，16H 为偏移量，其汇编形式中的 $0x80492e0$ 是转移目的地址，因此假定后面的 mov 指令的地址为 x，则满足：
 $0x80492e0 = x + 0x16$ ，故 $x = 0x80492e0 - 0x16 = 0x80492ca$ 。

(4) jmp 指令中的 E9H 为操作码，后面 4 个字节为偏移量，因为是小端方式，故偏移量为 $FFFFFF00H$ ，即 $-100H = -256$ 。后面的 sub 指令的地址为 $0x804829b$ ，故 jmp 指令的转移目标地址为 $0x804829b + 0xFFFFF00 = 0x804829b - 0x100 = 0x804819b$

- ⑨ 对于以下 x86-64 中的 AT&T 格式汇编指令，根据操作数的长度确定对应指令助记符中的长度后缀，并说明每个操作数的寻址方式。

```
(1) mov 8(%rbp, %rbx, 4), %ax
(2) mov %al, 12(%rbp)
(3) add (%rbp, %rsi, 4), %r9w
(4) or (%rbx), %dil
(5) sub %bpl, 8(%rsp, %rdi, 4)
(6) mov $0xFFFF0, %eax
(7) test %r8d, %r8d
(8) lea 8(%rbx, %rsi), %rax
(9) xor $0xF0, %rax
```

P120 第9题 参考答案:

- (1) 后缀:w, 源: 基址+比例变址+偏移 目的: 寄存器
 (2) 后缀:b, 源: 寄存器 目的: 基址+偏移
 (3) 后缀:w, 源: 基址+比例变址 目的: 寄存器
 (4) 后缀:b, 源: 基址 目的: 寄存器
 (5) 后缀:b, 源: 寄存器 目的: 基址+比例变址+偏移
 (6) 后缀:l, 源: 立即数 目的: 寄存器
 (7) 后缀:l, 源: 寄存器 目的: 寄存器
 (8) 后缀:q, 源: 基址+变址+偏移 目的: 寄存器
 (9) 后缀:q, 源: 立即数 目的: 寄存器

- ⑩ 假设 x86-64 系统某程序中变量 x 和 ptr 的类型声明如下:

```
src_type x;
dst_type *ptr;
```

这里, src_type 和 dst_type 是用 typedef 声明的数据类型。有以下一个 C 语言赋值语句:

```
*ptr=(dst_type) x;
```

若 x 存储在寄存器 RAX 或 EAX 或 AX 或 AL 中, ptr 存储在寄存器 RDX 中, 则对于表 3.13 给出的 src_type 和 dst_type 的类型组合, 写出实现上述赋值语句对应的汇编指令。

表 3.13 题 10 表

src_type	dst_type	汇编指令 (AT&T 格式)
char	long	
int	char	
int	unsigned long	
short	int	
unsigned char	unsigned	
char	unsigned long	
unsigned long	int	

P121 第10题 参考答案:

注: MOVSB, MOVZB 目的操作数只能是寄存器。

- (1) movsbq %al, %rbx
 movq %rbx, (%rdx)
 (2) movb %al, (%rdx)
 (3) movl %eax, %ebx #没有movzlb, 使用movl可以使高32位为0
 movq %rbx, (%rdx)
 (4) movswl %ax, %ebx
 movl %ebx, (%rdx)
 (5) movzbl %al, %ebx
 movl %ebx, (%rdx)
 (6) movzbq %al, %rbx
 movq %rbx, (%rdx)
 (7) movl %eax, (%rdx)

第四章

24 以下是结构体 test 的声明:

```
struct {
    char    c;
    double  d;
    int     i;
    short   s;
    char    *p;
    long    l;
    long long g;
    void    *v;
} test;
```

从开始, 相对地址



假设在 Windows 平台上编译, 则这个结构体中每个成员的偏移量是多少? 结构体总大小为多少字节? 如何调整成员的先后顺序使结构体所占空间最小?

P179 第22题 参考答案:

Windows平台要求不同的基本类型按照其数据长度进行对齐, 每个成员的偏移量如下:

c d i s p l g v

0 8 16 20 24 28 32 40

结构总大小为48字节, 因为其中的d和g必须是按8个字节边界对齐, 所以必须在末尾再加上4个字节, 即44+4=48字节。

变量按从大到小顺序排列, 可以使得结构所占空间最小, 因此调整顺序后的结构定义如下:

```
struct{
    double    d;
    long long g;
    int       i;
    char      *p;
    long      l;
    void      *v;
    short     s;
    char      c;
} test;
```

每个成员的偏移量如下:

d g i p l v s c

0 8 16 20 24 28 32 34

P179 第22题 类似参考题:

在x86-64中, 定义如下 C 结构体与指针变量:

```
struct S{
    short id;
    int total;
    char name[8];
} student[100];
```

偏

2

4

8

0

4

8

0000 B0D4

↑+4

student[1] = 0000 B0C0H + 16 = 0000 B0D0H

假设student数组的首地址为0000 B0C0H, student[1].total=0x1234 5678, 则56H所在单元地址是多少?

D4 78

D5 66

D6 34

D7 12

=> 0000 B0D5

0 1 | 2 3 | 4 5 6 7 | 8 9 10 11 12 13 14 15

short | int | char

P179 第22题 类似参考题:

在x86-64中, 定义如下 C 结构体与指针变量:

```
struct S{
    short id;
    int total;
    char name[8];
} student[100];
```

假设student数组的首地址为0000 B0C0H, student[1].total=0x1234 5678, 则56H所在单元地址是多少?

边界对齐, 小端排序, id之后补2个空字节, 所以

结构体总体大小为2+2+4+8=16个字节,

因此student[1]的首地址为: 0000 B0C0+16=0000 B0D0, total为第二个成员, 地址为 0000 B0D0+4=0000 B0D4,

答案为: 0x0000 B0D5

P178 第19题 参考答案:

19. 假设结构体类型 node 的定义、函数 np_init() 部分 C 语言代码及对应的 IA-32 部分汇编代码如下所示:

struct node{ int *p; struct { int x; int y; }s; struct NODE *next; }	void np_init(struct node *np) { np->s.x = ①; np->p = ②; np->next = ③; }	movl 8(%ebp), %eax movl 8(%eax), %edx movl %edx, 4(%eax) leal 4(%eax), %edx movl %edx, (%eax) movl %eax, 12(%eax)
---	--	--

(1) 结构体 node 所需存储空间为多少个字节? 成员 p、s.x、s.y 和 next 的偏移地址分别是多少?

(2) 根据最右侧的汇编代码填写函数 np_init() 缺失的表达式。

(1) 所需字节数为 4+(4+4)+4=16 字节。

成员 p、s.x、s.y 和 next 的偏移地址分别是 0、4、8、12。

(2)

① np->s.y

② &(np->s.x)

③ np